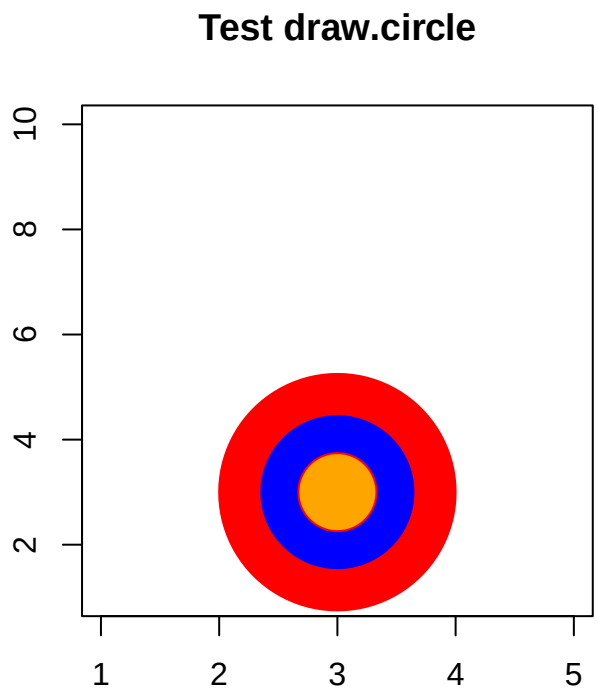
Introduction to Data Science
R Tutorial
Week 2

Evangelos Dimitriou and Dr. Fintan Nagle (inspired by Ilse Cuevas Andrade and Dr. Gah-Yi Ban)
Imperial College Business School

1. Plotting graphs and making functions

The aim of this exercise is to understand how functions work in R, how to modify them to accept new arguments, and how to plot shapes using the `plotrix` package. We'll start with a basic example that draws a circle and then make it more flexible.

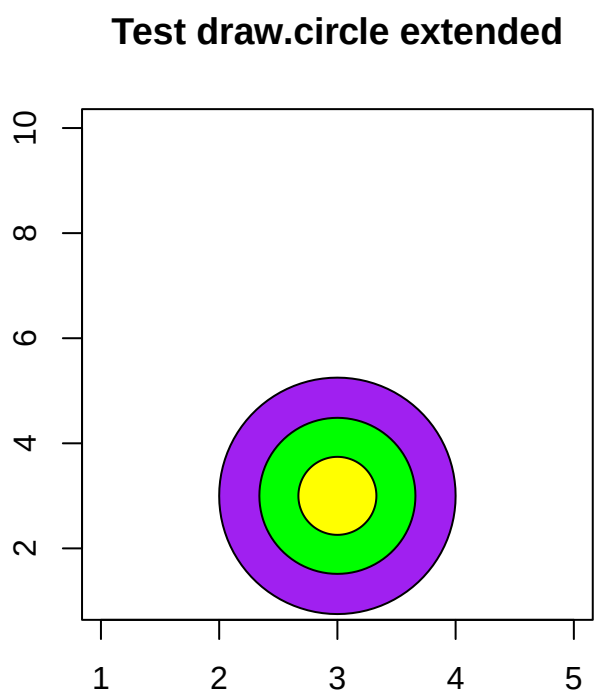
```
circle <- function(x,y){
  draw.circle(x,y,c(1,0.66,0.33),border="red",
  col=c("red","blue","orange"),lty=1,lwd=1)
}
library(plotrix)
par(pty="s")
plot(1:5,seq(1,10,length=5),type="n",xlab="",ylab="",main="Test draw.circle") # create empty plot
circle(3,3)
```



- Extend the function so that you can also pass in the colour you would like to apply to the circle.
- Where is the function call?
- When does draw.circle(... code actually gets executed?

```
# You can use ?draw.circle to see the available arguments for this function.
?draw.circle

# Define an extended version of our function:
circle_ext <- function(x,y,
                       border_colour = 'red', # default border color
                       colour = c('red', 'blue', 'orange')){ # default fill colors
  # When circle_ext() is called, this line is executed immediately.
  # The values of 'border_colour' and 'colour' depend on what was passed
  # in the function call. If nothing was passed, the default values above are used.
  draw.circle(x,y,c(1,0.66,0.33),
              border=border_colour, # border color passed by the user
              col=colour, # fill colors passed by the user
              lty=1,lwd=1)
}

library(plotrix) # ensure the package is loaded

par(pty = "s")  # keep the plotting region square-shaped

# Create a new empty plot again.
```

```r
plot(1:5, seq(1, 10, length = 5),
     type = "n", xlab = "", ylab = "",
     main = "Test draw.circle extended")

# This is the *function call* - where the function is actually executed.
# At this moment, R substitutes x=3, y=3, border_colour='black',
# and colour=c('purple','green','yellow'), and runs the code inside.
circle_ext(3, 3,
           border_colour = 'black',
           colour = c("purple", "green", "yellow"))
```



2. Deal with NAs

Real-world datasets often contain missing values, which in R are represented as `NA`. In this exercise, you'll learn how to identify and remove missing data using R's built-in functions.

1. Load the data from "`uk_gdp_and_weeklyWage_na.csv`".
2. Check for missing values (NAs) in the dataset.
3. Remove rows containing missing values using the function `na.omit()`.

Step 1: Set you working directory

- The working directory is the folder where R will look for files by default.

- Use setwd() to change it to the folder that contains your dataset.
- Tip: You can also use Session > Set Working Directory > Choose Directory.

```
setwd("~/Documents/Intro to Data Science - Imperial/Week 2")
```

Step 2: Load the dataset

- `read.csv()` reads data stored in a CSV (comma-separated values) file.
- The result is a data frame, which you can inspect using `head()` or `View()`.

```
uk_gdp_and_weeklyWage_na <- read.csv("data/uk_gdp_and_weeklyWage_na.csv")
```

Let's inspect the first few rows to see what the data look like:

```
head(uk_gdp_and_weeklyWage_na)
```

```
##   Year Weekly_pay   GDP_m
## 1 2000        296 1627447
## 2 2001        311 1662558
## 3 2002        323 1691998
## 4 2003        334 1744840
## 5 2004        346 1785756
## 6 2005        360 1833406
```

Optional: check the structure (types of columns)

```
str(uk_gdp_and_weeklyWage_na)
```

```
## 'data.frame':    23 obs. of  3 variables:
##  $ Year      : int  2000 2001 2002 2003 2004 2005 2006 2007 2008 2009 ...
##  $ Weekly_pay: int  296 311 323 334 346 360 374 390 405 412 ...
##  $ GDP_m     : int  1627447 1662558 1691998 1744840 1785756 1833406 1873015 NA 1918064 1831550 ...
```

Step 3: Identify missing values

Check how many missing values there are in total:

```
sum(is.na(uk_gdp_and_weeklyWage_na))
```

```
## [1] 6
```

You can also see which columns contain missing values:

```
colSums(is.na(uk_gdp_and_weeklyWage_na))
```

```
##       Year Weekly_pay      GDP_m
##          0          3          3
```

These functions return logical (TRUE/FALSE) indicators of whether a value is missing.

Step 4: Remove rows with missing data

`na.omit()` removes all rows that contain at least one `NA` value.

```
uk_gdp_and_weeklyWage_no_NA <- na.omit(uk_gdp_and_weeklyWage_na)
```

Check how many rows were removed:

```
nrow(uk_gdp_and_weeklyWage_na) - nrow(uk_gdp_and_weeklyWage_no_NA)
```

```
## [1] 5
```

Confirm that there are no missing values left:

```
sum(is.na(uk_gdp_and_weeklyWage_no_NA))
```

```
## [1] 0
```

Now uk_gdp_and_weeklyWage_no_NA contains only complete cases (no missing values).

3. Functions in R.

Write a function compute_s_n that for any given n computes the sum $S(n) = 1^2 + 2^2 + 3^2 + ...n^2$. Report the value of the sum when $n = 10$.

There are two approaches to solve this exercise.

1. First approach

```
compute_s_n_1 <- function(n) {
  sum_squares<-0 # initialize vector
  # using for loop

  for(i in 1:n){
    sum_squares <- sum_squares +(i^2)
  }

  return(sum_squares)
}
```

Let's see the if the function works

```
result1 <- compute_s_n_1(10)
print(result1)
```

```
## [1] 385
```

2. Second approach

```r
compute_s_n_2 <- function(n) {
  # using vectorized approach
  sum_squares <- sum((1:n)^2)

  return(sum_squares)
}
```

Let's see if the second approach works and gives the same result as the first one

```r
result2 <- compute_s_n_2(10)
print(result2)
```

```
## [1] 385
```

Exercise 1: Calculating GDP growth rate and classifying weekly wage levels

In this exercise, we'll work with the dataset `uk_gdp_and_weeklyWage.csv`

Our goal is to:

1. Load the dataset into R.
2. Add two new variables using `mutate()` from the `dplyr` package:

- `GDP_growth_rate`: the year-on-year percentage change in GDP.
- `weeklyWageLevel`: a logical variable indicating whether the weekly wage is greater than £500.

Some background knowledge

Gross Domestic Product (GDP) GDP measures the total value of everything a country produces in a year. If GDP increases, the economy has grown; if it decreases, the economy has shrunk.

GDP Growth Rate The growth rate tells us how much GDP has changed compared to the previous year. It's computed using the formula:

$$\text{GDP growth rate} = \frac{\text{GDP this year} - \text{GDP last year}}{\text{GDP last year}} \times 100$$

After loeading the datasets we are ready to create the new columns

```r
# Step 1: Load the dataset
# Make sure "uk_gdp_and_weeklyWage.csv" is saved in the "data" folder
uk_gdp_and_weeklyWage <- read.csv("data/uk_gdp_and_weeklyWage.csv")

# Step 2: Add two new columns
uk_gdp_and_weeklyWage <- uk_gdp_and_weeklyWage %>%
  mutate(
    # GDP_growth_rate: how much GDP changed since last year, in %
    # lag(GDP_m) gets the previous year's GDP, you can use the help window to find more details
    GDP_growth_rate = (GDP_m - lag(GDP_m)) / lag(GDP_m) * 100,
```

```
    # weeklyWageLevel: TRUE if Weekly_pay > 500, FALSE otherwise
    weeklyWageLevel = Weekly_pay > 500
  )
```

Let's have a look at the result

```
head(uk_gdp_and_weeklyWage)
```

```
##   Year Weekly_pay   GDP_m GDP_growth_rate weeklyWageLevel
## 1 2000        296 1627447              NA           FALSE
## 2 2001        311 1662558        2.157428           FALSE
## 3 2002        323 1691998        1.770765           FALSE
## 4 2003        334 1744840        3.123053           FALSE
## 5 2004        346 1785756        2.344971           FALSE
## 6 2005        360 1833406        2.668338           FALSE
```

Exercise 2 (Challenging!): Working with Leap Years

Goal

In this exercise, we'll learn how to: - Loop through rows in a dataset - Apply **conditional logic** to identify **leap years** - Build a new dataset (`data.frame`) that only contains the leap years

---

Background: What is a Leap Year?

A **leap year** is a year with an extra day in February (29 days instead of 28).
This keeps our calendar year synchronized with the Earth's orbit around the Sun.

To determine whether a year is a leap year, we use the following rules:

1. The year must be **divisible by 4**
   → e.g., 2016, 2020, 2024
2. But if the year is also **divisible by 100**, it is **not** a leap year
   → e.g., 1900, 2100
3. However, if it is **divisible by 400**, it **is** a leap year
   → e.g., 2000, 2400

Step-by-step in R

Step 1: Initialize an empty dataset We will store all leap year rows here

```
leap_year_subset <- NULL
```

Step 2: Loop through each row of the dataset `nrow()` returns the total number of rows in `uk_gdp_and_weeklyWage`

```r
for (i in 1:nrow(uk_gdp_and_weeklyWage)) {

  # Extract the year value for the current row
  year_num <- uk_gdp_and_weeklyWage$Year[i]

  # Step 3: Apply leap year rules using nested if-else statements

  # Case 1: If the year is divisible by 100, check if it is also divisible by 400
  if (year_num %% 100 == 0) {

    # If divisible by 400, then it is a leap year
    if (year_num %% 400 == 0) {
      leap_year_subset <- rbind(leap_year_subset, uk_gdp_and_weeklyWage[i, ])
    }

  # Case 2: Otherwise (not divisible by 100), check if divisible by 4
  } else if (year_num %% 4 == 0) {

    # If divisible by 4, it's a leap year
    leap_year_subset <- rbind(leap_year_subset, uk_gdp_and_weeklyWage[i, ])

  # Case 3: If none of the above are true, it's not a leap year
  } else {
    print(paste(year_num, "is not a leap year"))
  }
}
```

```
## [1] "2001 is not a leap year"
## [1] "2002 is not a leap year"
## [1] "2003 is not a leap year"
## [1] "2005 is not a leap year"
## [1] "2006 is not a leap year"
## [1] "2007 is not a leap year"
## [1] "2009 is not a leap year"
## [1] "2010 is not a leap year"
## [1] "2011 is not a leap year"
## [1] "2013 is not a leap year"
## [1] "2014 is not a leap year"
## [1] "2015 is not a leap year"
## [1] "2017 is not a leap year"
## [1] "2018 is not a leap year"
## [1] "2019 is not a leap year"
## [1] "2021 is not a leap year"
## [1] "2022 is not a leap year"
```

Step 4: Check the results Display the resulting subset of leap years

```r
leap_year_subset
```

```
##    Year Weekly_pay   GDP_m GDP_growth_rate weeklyWageLevel
## 1  2000        296 1627447              NA           FALSE
## 5  2004        346 1785756       2.3449715           FALSE
```

```
## 9  2008        405 1918064      -0.1543444           FALSE
## 13 2012        434 1923551       1.4484567           FALSE
## 17 2016        464 2114406       2.1652062           FALSE
## 21 2020        516 1991439     -11.0308585            TRUE
```

##The end